Tarea 2 RA1

Random Access File

Objetivos de la práctica

- Escribir y leer datos binarios con RandomAccessFile.
- Diferenciar acceso secuencial y acceso directo.
- Diseñar registros de tamaño fijo.
- Modificar solo un campo sin sobrescribir el resto del registro.
- Construir un índice auxiliar para saltar directamente a un registro.

Ejercicio 1. Fichero aleatorio de empleados

Objetivo. Practicar la escritura de registros fijos con varios tipos de datos y asociar cada empleado a una posición calculable dentro del fichero.

Estructura de datos

Campo	Tipo	Tamaño
Identificador	int	4 bytes
Apellido	10 caracteres Unicode	20 bytes
Departamento	int	4 bytes
Salario	double	8 bytes
Total del registro	-	36 bytes

Enunciado.

Crea un programa que inserte datos de empleados en un fichero aleatorio.

Para cada empleado se debe guardar: identificador, apellido, departamento y salario.

El identificador debe ser mayor que cero y debe coincidir con el índice + 1 que recorren los arrays de ejemplo.

Escribe varios empleados en el fichero y comprueba que cada registro se almacena siempre con el mismo tamaño.

Al terminar, recorre el fichero y muestra todos los registros para verificar que se han escrito correctamente.

Ejercicio 2. Escritura y lectura secuencial de enteros

Objetivo. Entender el uso básico de RandomAccessFile para escribir datos binarios y volver al inicio del fichero con seek(0).

Enunciado.

Crea un programa que abra o cree el archivo nums.bin en modo lectura/escritura.

Pide por pantalla los 5 números enteros favoritos del usuario y escríbelos en el fichero.

Cuando termines de escribir, vuelve el puntero al inicio y lee los 5 enteros mostrando cada valor por consola.

Utiliza únicamente RandomAccessFile y sus métodos básicos.

Indica también qué valor devuelve getFilePointer() justo después de escribir los 5 enteros.

Ejercicio 3. Acceso directo a slots de 4 bytes

Objetivo. Calcular offsets a partir del tamaño de un entero para leer una posición concreta sin recorrer las anteriores.

Enunciado.

Crea un archivo llamado slots.bin.

Escribe los enteros del ejercicio anterior de forma que cada uno quede colocado en su propio slot de 4 bytes.

Lee solamente el segundo valor sin leer los demás y muéstralo por consola.

Explica brevemente cómo se calcula la posición de lectura a partir del número de slot.

Ejercicio 4. Cadenas cortas con writeUTF/readUTF

Objetivo. Diferenciar las cadenas grabadas con egundo e variable de los registros de tamaño fijo.

Enunciado.

Crea el egundo nombres.bin y escribe tres nombres cortos usando writeUTF.

Vuelve al inicio del fichero con seek(0) y recupéralos con readUTF, mostrando cada nombre en una línea.

Explica por qué no puedes saltar directamente al egundo nombre sin leer antes el primero.

Ejercicio 5. Registros fijos básicos: id y nombre

Objetivo. Diseñar un registro simple de tamaño fijo y usar ese tamaño para ir directamente al registro n.

Enunciado.

Define un registro con id (int, 4 bytes) y nombre (10 caracteres, 20 bytes con writeChar).

Escribe tres registros de ejemplo en un fichero binario.

Lee directamente el tercer registro y muestra sus campos por consola.

Ejercicio 6. Actualización de un campo dentro de un registro fijo

Objetivo. Modificar solo un dato concreto aprovechando que todos los registros ocupan el mismo número de bytes.

Enunciado.

Crea personas.dat en modo "rw". Si el archivo está vacío, escribe al menos 3 registros de ejemplo.

Pide por consola un id existente y una nueva edad.

Localiza el registro con ese id recorriendo secuencialmente el fichero.

Si el id coincide, quédate en ese registro; si no coincide, salta al siguiente usando seek(reglnicio + 48).

Una vez localizado el registro correcto, mueve el puntero hasta el campo edad y actualízalo sin sobrescribir id ni nombre.

Vuelve al inicio de ese mismo registro y muéstralo completo para verificar el cambio.

Ejercicio 7. Índice por id en archivo aparte

Objetivo. Crear un índice auxiliar para evitar recorrer siempre el fichero principal completo.

Estructura de datos

Fichero	Contenido	Tamaño por entrada
personas.dat	Registro fijo id + nombre + edad	48 bytes
personas.idx	Par (id:int, offset:long)	12 bytes

Enunciado.

Recorre personas.dat y construye un segundo fichero llamado personas.idx.

Por cada registro situado en la posición i, escribe en el índice el par (id, i * 48L).

Implementa una búsqueda por id abriendo personas.idx y recorriéndolo de forma lineal.

Cuando encuentres el id buscado, usa el offset recuperado para saltar a personas.dat y leer solo ese registro.

Muestra por consola el registro encontrado y el offset utilizado.

Entrega sugerida

- Un proyecto Java con un menú o una clase principal por ejercicio.
- Salidas por consola que permitan comprobar puntero, offsets y datos leídos.
- Comentarios mínimos solo donde ayuden a entender cálculos de posiciones.
- Un breve apartado final explicando la diferencia entre datos de tamaño fijo y variable.